Now let's edit your .env/production/database file to be as follows:

```
POSTGRES_USER=postgres
POSTGRES_PASSWORD=my-production-password
POSTGRES_DB=myapp_production
```

We've changed the Postgres password so that it's different from development. We've also updated the database name to be myapp_production instead of myapp_development.

Although there's more that could be tweaked, we've completed the essential configuration for production.

A Production Image: Precompiling Assets

In development, by default, Rails compiles our assets for each request so that our changes are picked up automatically. However, typically in production, we precompile our assets once and then serve them up as static files for faster load times. Rails provides the following Rake task for this:

```
bin/rails assets:precompile
```

Up until now, the changes we've needed to make for our app to run in production have just been config changes, or tweaks that would be fine in development too. Here, however, the production version of our app needs additional files: the compiled assets.

How do we achieve this with our Docker setup?

The solution is to create a second, production-flavored image that precompiles the assets at build time, so the compiled assets are baked into the image itself. Generally, it's a good idea to keep your development environment as similar to production as possible. However, some changes, like this need to precompile assets for production, require that our development and production environments diverge slightly.

Let's create a Dockerfile for our production image. Start by taking a copy:

```
$ cp Dockerfile Dockerfile.prod
```

Next, let's update our Dockerfile.prod to precompile assets by adding the following line just before the ENTRYPOINT command:

```
RUN bin/rails assets:precompile
ENTRYPOINT ["./docker-entrypoint.sh"]
CMD ["bin/rails", "s", "-b", "0.0.0.0"]
```